

Distributed Deep Learning Inference at Edge Devices

FYP Project Proposal - 16 Intake

160170B	K.M.B.G.B Gunarathne
160477X	M.V.R.Perera
160628M	T.M.C.P Thennakoon

Table Of Content

Table Of Content	1
1. Introduction	2
2. Problem Statement	4
3. Research Objectives	6
4. Literature Review	7
5. Methodology	10
5.1 Background of DNN	10
5.2 Fully Connected Neural Network	10
5.3 Model Parallelism	12
5.4 Communication and Task sharing	12
6. Research Timeline	14
7. Conclusion/Summary	14

1. Introduction

Deep neural networks are becoming an important tool in modern computing applications. Deep/Convolutional Neural Networks (DNNs & CNNs) is a heavily researched domain and it is widely used in immense data processing applications due to their higher accuracy. In this context there are specialized deep neural networks introduced for various tasks. However there is a requirement of a lot of data and computation power to train and use these models.

Edge computing and IoT applications have emerged with the requirement of collecting and processing large sets of data locally within devices. Thus ensuring privacy and low latency. However these edge devices are resource constrained and have less computational power compared to modern computers and servers. These edge devices may collect large sums of sensory data where it needs to be processed and get results in real-time for mission critical applications. Deploying Deep/Convolutional Neural Networks (DNNs & CNNs) applications in these mission critical resource constrained devices is a challenging task due to discussed conditions above. DNN/CNN has two phases which are called training and inference. Training of a neural network requires a large set of data and huge computational effort. Once the model is trained and ready it can be provided previously unseen/untrained input data and will return results of identified or classified patterns in data. This phase is called inference. Even Though it does not require huge computational power compared to training, executing inference tasks in resource constrained edge devices is a challenging task.

Furthermore performing inference tasks of deep learning applications on edge devices ensure privacy of input data and can result in shorter latency when compared to a cloud solution. As most devices are memory constrained and compute constrained, they cannot store and execute a complete Deep Neural Network(DNN). One possible solution is to distribute the DNN across multiple edge devices. Layers in a neural network need to be carefully partitioned to balance the amount of computation and memory usage on each resource constrained edge device and reduce communication among devices.

In this research we are focusing on the execution of the DNN inference part in resource constrained edge devices like Raspberry Pi drives. We are intended to research and propose a novel approach to partition and distributedly execute inference calculation on multiple edge devices which are connected in a local network. Furthermore we plan to investigate how to efficiently reduce the communication demands among the devices.

2. Problem Statement

IoT devices have been facing this issue of low resources and low computational power since the beginning of the IoT era[10]. In some IoT applications that need to use immense technologies like augmented reality, video processing, cognitive assistance and machine vision will be unable to get best performance and real time experience with these constraints (e.g., computation, memory, energy).

Performing inference tasks of deep learning applications on IoT edge devices requires large computational power and memory space which are usually limited on IoT devices like Raspberry Pis. These devices have limited resources eg: 1GB - 2GB RAM , 1GHz - 1.5GHz processor etc. (Depends on the versions.) Therefore storing and execution of a large complete Deep Neural Network in a single device is an unfeasible task. Even a medium scale DNN may present intolerable delays in executing inference calculations in these devices. Most common solution to this problem is cloud based computation which introduces availability, privacy and latency issues. Therefore by computing it locally ensures the privacy of sensitive data and results in low latency.

When there are a large number of layers and also when there are a large number of neurons in one layer of a fully connected neural network or when there are computational intensive convolutional layers. The resources of a Raspberry Pi device will not be enough or it will take unacceptable time to compute the result.

Board	MobileNet v1(ms) Benchmark value	MobileNet V2(ms) Benchmark value	Inception model that we tested(s)
Raspberry Pi(TF Lite)	271.5	379.6	24
Raspberry Pi(TF)	480.3	654.0	32

Figure 1.

As shown in the above table (Figure 1.), Tensorflow optimised models like MobileNet v1 and v2 have acceptable low time consumption to complete execution but we have tested a custom model (Inception model) (Raspberry pi 3model B V1 2)- a object detection library on our Raspberry Pi 3 B setup and it took unacceptable time consumption compared to the Tensorflow given benchmark values[11].

We identified the lack of conducted research in this area to present model partitioning and distributed execution in multiple Raspberry Pi devices considering computational , memory and communication demands. Thus make it possible to run large deep neural networks and bring overall performance improvement of inference execution.

3. Research Objectives

The final intended outcome of this research will be to propose a novel approach to build communication and memory aware partitioning and distribution schemes for fully connected neural network layers of deep neural networks among Raspberry Pi devices. Furthermore the proof of concept prototype for distributed DNN/CNN execution is planned to be implemented.

- a. Discover model parallelism based partitioning scheme for fully connected neural networks.
- b. Propose task distribution and task sharing mechanism among Pi devices.
- c. Implement proof of concept DNN/CNN partitioning and distributed processing application using Cpp

4. Literature Review

Improving Deep Learning inference performance to overcome challenges it has faced is an intensively researched area. Inference is computationally costly when it comes to high dimensional inputs. Therefore a lot of calculations have to be done on that input data. One common solution was to use cloud computing as a solution, as they can use cloud resources for computation. But moving the data from source to cloud introduced some challenges such as latency, scalability and privacy. When it comes to real time applications, latency incurred by additional queuing and propagation delays of the network is not acceptable. Unpredictability of the remote cloud server status is also a problem. Uploading all the data to the cloud is inefficient too. Sometimes there may be privacy issues when uploading data. Therefore to avoid those challenges edge computing came into the play. Edge computing reduced the latency by doing computations in less proximity to the end devices. A hierarchical structure and a local edge server can be used to avoid the privacy issues. But edge computing also introduced new challenges. One of the main challenges is using the less powerful edge computing devices for the high resource requirements in deep learning applications. An edge device may be an edge server with a GPU or a mobile device or it may be even a Raspberry Pi device. Another challenge was to coordinate with edge devices to produce an output.

According to paper [1] they have mainly identified three types of solutions for use in edge computing with Deep learning inference.

1. All computations done on the end device.
2. Edge server computation.
3. Computation across edge devices.

In the first method, the model is located at the edge device. As the edge devices are not capable of doing heavy computation and memory intensive tasks, those models need to be designed with less parameters. Otherwise later the parameters should be compressed or unnecessary parameters should be removed. In DeepIoT [2], the authors present a pruning method for DNN models used in IoT devices. But this may have an impact by reducing the

model accuracy. If not the above methods are used, application specific hardware changes are used as research solutions.

Though all those methods are considered, the first method is still challenging because of the resource limitations such as power, computation and memory. Because of that some researchers tend to use the second method which involves an edge server. As done in [3] deep learning framework, where they offload the whole DNN to the edge server for computations. But sometimes it is helpful to do data preprocessing to reduce the communication time with the edge server. In [4], they use removing blurry images, and cropping to take only the required parts of images as preprocessing techniques. A main problem faced here is resource management in the edge server. As multiple end devices may be using the same shared edge server. Because of that tradeoff between latency, accuracy and other performance metrics need to be considered. Some solutions such as Mainstream[5], use transfer learning, where common lower layers of the DNN model is used for multiple applications, to achieve the above mentioned tradeoff.

Although the second method increases the speed of inference, there is a third method more efficient than the above method, where intelligent offloading is considered. The solution we propose is related to this method. Solutions done in this method, faces the challenges such as taking a decision whether to offload the entire DNN or not (depends on size of the data, hardware abilities, DNN model, network quality), which fractions of the DNN computations should be offloaded. When using DNN partitioning, some solutions have used layer-wise partitioning as in Neurosurgeon[6] and some have used input-wise partitioning as in MoDNN[7] and DeepThings[8]. Input-wise partitioning also has a problem of increased data dependency. Some solutions such as DDNN[9] are using a hierarchy of cloud, edge servers and end devices to model partitioning. Cloud is used only if total processing time is reduced, because of the available powerful resources and no privacy risk. We are using a distributed computation method across multiple helper edge devices. Same as in MoDNN[7] and DeepThings[8], we will be using fine-grained partitioning of the DNN executions. Our solution will be presented using Raspberry Pi devices as edge devices. In our research we are planning to improve our solution with more efficient load balancing heuristics and dynamically assigning the data to edge devices, considering resource availability and network

conditions. We will be focusing on reducing the latency through those ways, as current state of the art still results in a quite high latency.

5. Methodology

5.1 Background of DNN

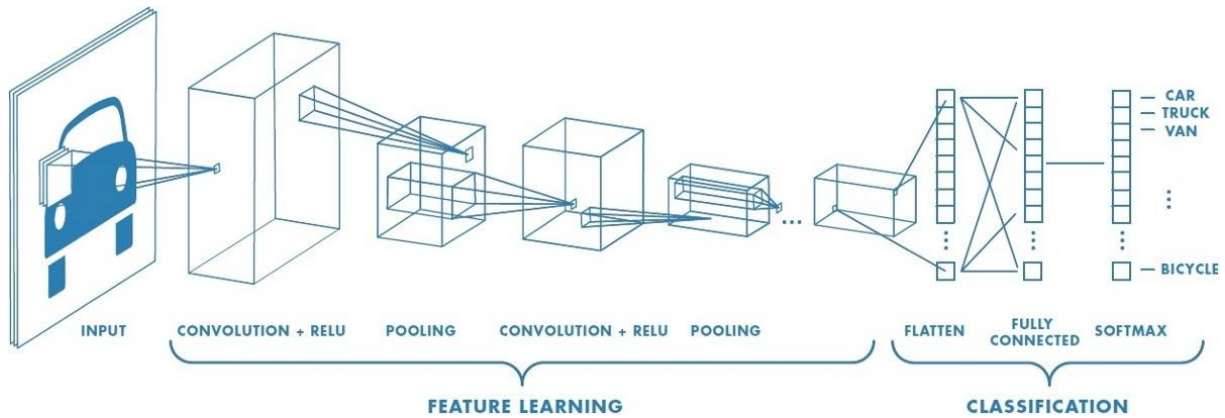


Figure 2.

The Figure 2 shows an overview of an Image classification Deep Neural Network with convolutional layers and fully connected layers. Convolutional layers are used to extract features (ie: edges, lines, curves etc) in the given input. The fully connected network is used to classify the image based on recognized features of the input data. Within these layers heavy computational tasks are executed while propagating through the network.

Within these convolutional layers it applies multiple pre learned filters to their input to produce a set of feature maps which extract some features from the input. There can be many filters between each convolution layer which increase the computational and memory demand. Pooling layers are used to shrink the size of input feature maps.

5.2 Fully Connected Neural Network

Fully connected neural networks are usually used for recognizing and classification tasks. Therefore convolution layers followed by fully connected layers are used to image recognition and classification. Fully connected layers consist of a number of neurons which

have a pre learned bias value and an activation function. These neurons are connected with all previous layer's neurons and the next layer's neurons where these connections have pre-learned weights.

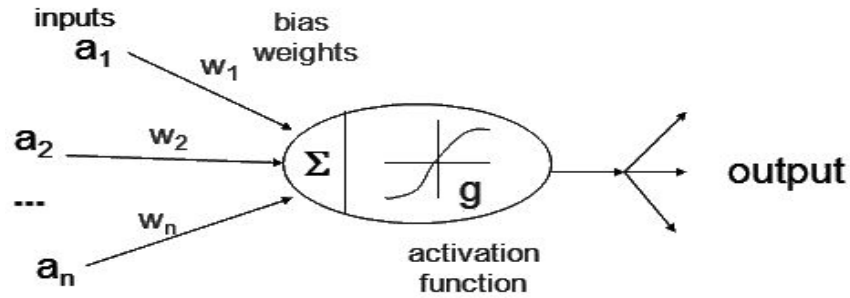


Figure 3.

The calculation happen inside each neurons can be described as follows

$$f\left(b + \sum_{i=1}^n x_i w_i\right)$$

Where

b = bias

x = input to neuron

w = weights

n = the number of inputs from the incoming layer

i = a counter from 0 to n

f = an Activation function

Therefore each calculation requires all input values of neurons for all connected none zero weights. Therefore one possible solution will be to partition each calculation of neurons by sharing all input data and weight data to all devices. However it will be not memory efficient as well as communication efficient.

5.3 Model Parallelism

Model parallelism is used to partition the neural network layers in vertical direction and assign each device with a subset of these neurons to execute calculations. As this method splits the neural network this method can be possibly applied to deploy large neural networks which normally can not deploy to a single device. Furthermore model parallelism is used for training large neural networks in GPU clusters. Therefore applying this method for the inference task is sound feasible and promising.

5.4 Communication and Task sharing

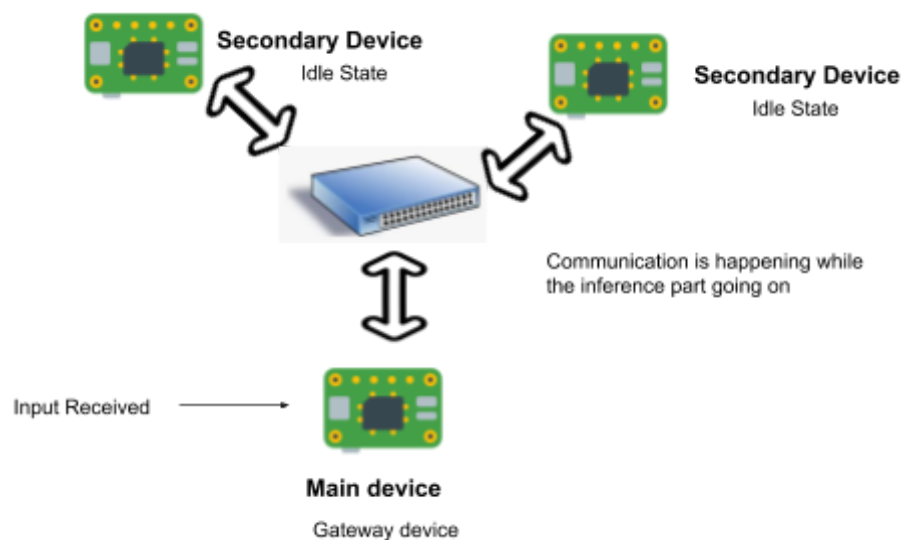


Figure 4.

As shown in the above figure (figure 4) the main device (gateway device) will receive input data through its sensors. In this case we assume that there are two other devices on the same network which are in idle state or doing less computationally expensive tasks. These secondary devices also can be changed to a main device (Gateway device) dynamically when it receives an input. All three devices have trained Image Calcification model and the partitioning and sharing services running in the background.

The main device which captured the image will run the partitioning program to partition the neural network and input data to separately executable tasks and it will add them to a task queue. Then the main device will start the inference execution by taking one task by one from the queue. Meanwhile it will broadcast a task sharing request among the network and secondary devices which are in idle state will capture this request and they will notify the main device that they are available to participate. The main device will then start sending a set of tasks from the queue to secondary devices. When the secondary devices have finished the calculation in each neuron they will update the main device with the results. And request more tasks from the queue until it is empty or the execution is interrupted due to its own requirements of executing its own process.

We are planning to implement the proposed system using C++ programming language as the C++ language is more compatible with low level devices like Raspberry Pis and it helps to gain faster execution time.

6. Research Timeline

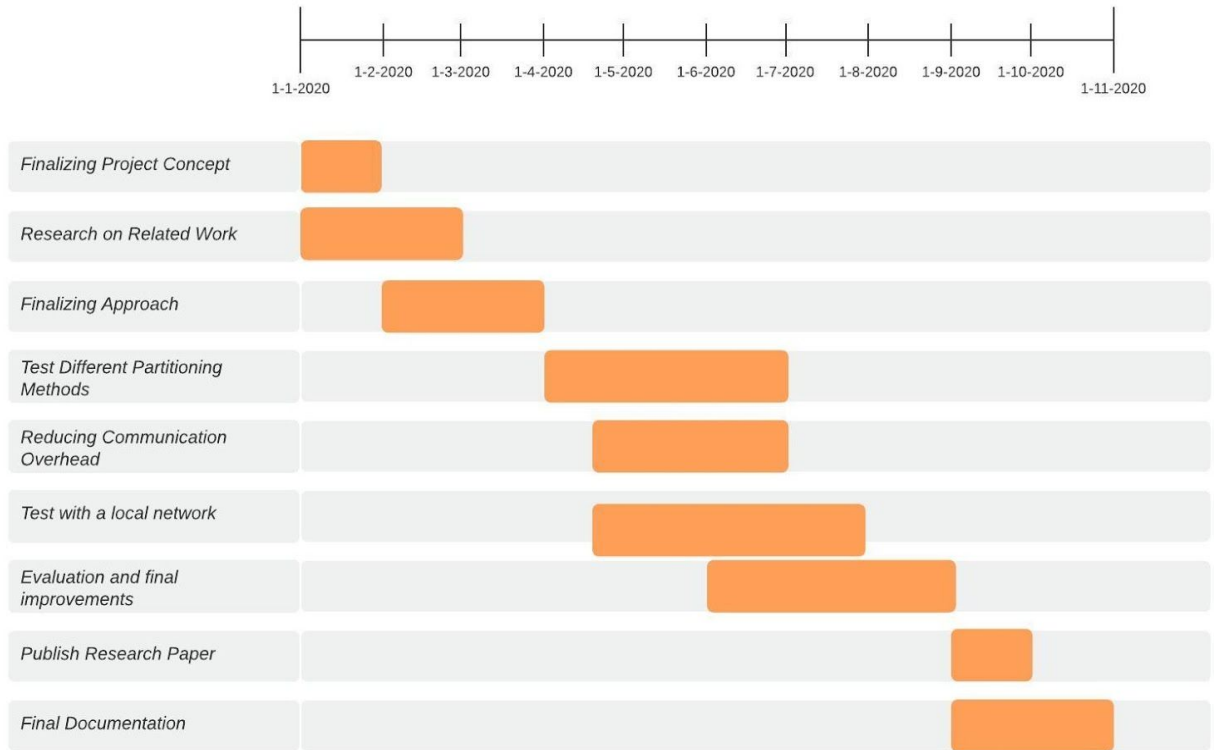


Figure 5. (Tentative)

7. Conclusion/Summary

In conclusion this project is intended to propose and develop novel applications that can be used to distributedly execute inference of fully connected neural networks. We hope that these distributed Raspberry Pi based partitioning methods will reduce the time taken for inference tasks and make it possible to run large neural networks which usually can not deploy to a single device. Furthermore we plan to investigate the ideal method for partitioning to reduce communication demand among these Raspberry Pis to prevent unwanted overheads. We hope this research will help to boost future IoT based and edge computing applications capabilities.

References

- [1] J. Chen and X. Ran, “Deep Learning With Edge Computing: A Review,” *Proc. IEEE*, vol. 107, no. 8, 2019.
- [2] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du, “DeepIoT: Compressing deep neural network structures for sensing systems with a compressor-critic framework,” in *Proc. SenSys*, 2017, pp. 1–4.
- [3] X. Wang, X. Wang, and S. Mao, “RF sensing in the Internet of Things: A general deep learning framework,” *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 62–67, Sep. 2018.
- [4] C. Liu et al., “A new deep learning-based food recognition system for dietary assessment on an edge computing service infrastructure,” *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 249–261, Jan. 2018.
- [5] A. H. Jiang et al., “Mainstream: Dynamic stream-sharing for multi-tenant video processing,” in *Proc. USENIX Annu. Tech. Conf. (USENIXATC)*, 2018, pp. 29–42.
- [6] Y. Kang et al., “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 615–629, 2017.
- [7] J. Mao, X. Chen, K. W. Nixon, C. Krieger, and Y. Chen, “MoDNN: Local distributed mobile computing system for deep neural network,” in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, Mar. 2017, pp. 1396–1401.
- [8] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2348–2359, Nov. 2018.

- [9] S. Teerapittayanon, B. McDanel, and H. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” in Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS), Jun. 2017, pp. 328–339.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” IEEE Internet of Things Journal (IoT-J), vol. 3, no. 5, pp. 637–646, 2016.
- [11] Leigh Johnson “Portable Computer Vision: TensorFlow 2.0 on a Raspberry Pi”online:<https://towardsdatascience.com/portable-computer-vision-tensorflow-2-0-on-a-raspberry-pi-part-1-of-2-84e318798ce9>.

Figures/Tables

Figure	Page No.
Figure 1 - Comparison of image calcification’s time taken in deference models	4
Figure 2 - CNN architecture	10
Figure 3 - Activate Function process inside a neuron	11
Figure 4 - Raspberry pi Communication custer	12
Figure 5- Research Timeline	14